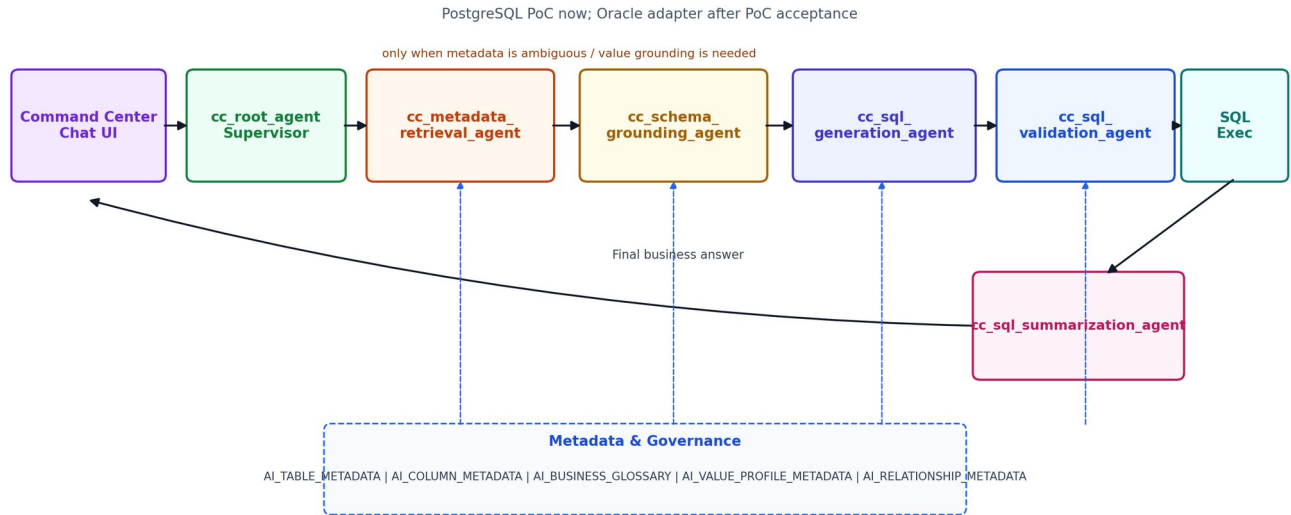


Command Center Agentic Text-to-SQL PoC

Implementation Playbook - Exact Agent, Tool, Flow and Metadata Changes

Current runtime: PostgreSQL | Future production target: Oracle Database

Target Pattern: Supervisor + Controlled Pipeline + Conditional Grounding



Why this revision exists

The earlier design explained the architecture but did not give enough implementation detail. This version is written as a build guide: for every agent and tool it states what it does, what to change, the exact inputs/outputs, prompt structure, implementation tasks, and acceptance checks. It also gives concrete PostgreSQL metadata enrichment steps that solve the natural-language-to-column-name problem.

1. What is wrong with the current PoC design

From the current agent configuration and prompts, the workflow is a fixed sequence: metadata retrieval -> exploratory metadata SQL generation -> exploratory metadata SQL execution -> final SQL generation -> validation -> execution -> summarization. The init pair is compensating for incomplete column semantics by querying sample values from a flattened metadata table.

- The root agent mainly lists a fixed sequence; it is not yet acting as a true supervisor that decides when grounding is needed.
- The metadata retrieval agent is a thin wrapper around `command_center_metadata_retrieval` and returns whatever the tool returns. It does not explicitly resolve business language to physical schema names.
- `cc_sql_init_generation_agent` and `cc_sql_init_execution_agent` are useful as a temporary workaround, but they create an extra LLM-generated SQL round trip just to inspect metadata values.
- `cc_sql_generation_agent` is being asked to compensate for weak metadata. That is why exact physical names work better than phrases such as “account number”, “created date”, “case status”, or “Tampa cases”.
- `cc_sql_validation_agent` currently focuses mainly on syntactic correctness. It needs deterministic allowlist/security validation as well.
- `cc_sql_execution_agent` is described as a SQL generation assistant even though it should only execute already-validated SQL.

- Because the PoC is PostgreSQL, ILIKE and LIMIT are valid now. Oracle syntax should be introduced only through a later dialect adapter.

Core design principle

Do not solve business-language schema matching inside the final SQL generator. Resolve the meaning first, then generate SQL from grounded table/column/value mappings.

2. Target design pattern

Use a Supervisor + Controlled Pipeline pattern. cc_root_agent remains mandatory and owns the workflow. Worker agents have narrow responsibilities and do not talk directly to the user. Schema grounding is conditional: it runs only when metadata confidence is low or a user value must be mapped to the correct column.

- cc_root_agent receives the user question and starts metadata retrieval.
- cc_metadata_retrieval_agent returns candidate tables/columns, semantic matches, verified relationships, confidence, and unresolved terms.
- If metadata is ambiguous or user values need location/category/status grounding, cc_root_agent calls cc_schema_grounding_agent. Otherwise it skips this step.
- cc_sql_generation_agent generates one PostgreSQL SELECT/CTE query using only grounded metadata.
- cc_sql_validation_agent performs deterministic and LLM-assisted validation. Invalid SQL is sent back for at most two regeneration attempts.
- cc_sql_execution_agent executes only validated SQL through command_center_run_sql.
- cc_sql_summarization_agent converts the SQL result into a concise business answer and returns it to cc_root_agent for the user.

Fallback if your orchestration engine only supports fixed sequencing

Keep cc_schema_grounding_agent in the fixed chain, but make it a no-op when retrieved_metadata.needs_grounding=false. This preserves the preferred contract without forcing every request to perform exploratory lookup.

3. Shared workflow state - standardize these output keys first

The current prompts already rely on output_key variables. Standardize the shared state so every agent has a predictable contract. This reduces prompt coupling and makes future Oracle migration easier.

Key	Produced by	Purpose
user_question	Chat / root	Original user request, never overwritten
retrieved_metadata	cc_metadata_retrieval_agent	Candidate tables, columns, glossary matches, relationships, confidence
grounding_output	cc_schema_grounding_agent	Resolved ambiguous columns and filter values; may be PASS_THROUGH
sql_generation_output	cc_sql_generation_agent	SQL, parameter map, tables/columns used, generation status
sql_validation_output	cc_sql_validation_agent	valid/invalid status, validated SQL, errors, warnings
sql_query_result	cc_sql_execution_agent	Rows, columns, row count, truncation flag, DB error
summary_response	cc_sql_summarization_agent	Final business answer
retry_count	cc_root_agent	Maximum 2 generation retries
database_dialect	cc_root_agent/config	postgresql during PoC; oracle after migration

4. Agent-by-agent implementation changes

4.1 cc_root_agent - change from static sequence to supervisor

What it should do: own the conversation, decide the next step, enforce retry limits, and ensure execution never happens before validation. It should not generate or execute SQL itself.

Exactly what to update

Replace the current “orchestrate these agents in sequence” instruction with decision rules. Add conditional grounding, retry control, and failure routing. The root agent should never directly call database tools.

- Input: user_question plus current workflow state.
- Tools: none. Its capability is agent orchestration, not direct DB access.
- If retrieved_metadata.needs_grounding=true, call cc_schema_grounding_agent.
- If metadata or grounding returns multiple plausible interpretations, the root is the only agent allowed to ask the user for clarification.
- If validation fails, send validation_errors + previous SQL back to cc_sql_generation_agent. Stop after 2 retries.
- If execution fails with a correctable SQL error, route through validation/generation; otherwise summarize the failure safely.
- Successful workflows must end in cc_sql_summarization_agent.

instruction: |-

```
You are the Command Center Root Agent and workflow supervisor.  
You do not generate SQL and you do not execute SQL.
```

Required order:

```
metadata retrieval -> optional schema grounding -> SQL generation  
-> SQL validation -> SQL execution -> summarization.
```

Rules:

- Never execute before validation returns is_valid=true.
- Invoke schema grounding only when retrieved_metadata.needs_grounding=true.
- Only you may ask the user a clarification question.
- On validation failure, retry SQL generation at most 2 times using validation_errors.
- Never invent tables, columns, relationships, or values.
- database_dialect is PostgreSQL during the PoC.

response_type: JSON

output_key: root_response

Acceptance test:

- A simple count question should skip grounding.
- A “Tampa” or ambiguous business-term question should invoke grounding.
- No path may call SQL execution before validation.
- The workflow stops after two failed generation retries.

4.2 cc_metadata_retrieval_agent - turn retrieval into schema linking

What it should do: map normal business language to likely physical schema objects before SQL generation. The tool must search enriched metadata, not just return raw table/column lists.

Exactly what to update

Keep the agent as a narrow tool wrapper, but upgrade command_center_metadata_retrieval and its output contract. The agent should return candidate mappings with confidence, unresolved_terms and needs_grounding. Do not force

it to return only an opaque raw blob.

- Tool: `command_center_metadata_retrieval` (existing, must be enhanced).
- Match in this order: exact physical name -> normalized physical name -> display name -> synonym/glossary -> description/full-text match -> optional semantic similarity.
- Return only the top relevant tables/columns, not the entire schema.
- Include verified relationships when more than one table is a candidate.
- Set `needs_grounding=true` when confidence is below the agreed threshold or when a literal user value could belong to multiple columns.
- Return `unresolved_terms` rather than silently guessing.

instruction: |-

You are the Command Center Metadata Retrieval Agent.

Your only job is to retrieve and rank schema metadata relevant to {user_question}.

Always call `command_center_metadata_retrieval`.

Return candidate tables, columns, descriptions, display names, synonyms, semantic types, value profiles, verified relationships, confidence scores, `unresolved_terms` and `needs_grounding`.

Do not generate SQL. Do not execute SQL. Do not invent metadata.

response_type: JSON

output_key: `retrieved_metadata`

Example `retrieved_metadata`:

```
{
  "tables": [{"table_name": "PACT_WORK", "confidence": 0.94}],
  "columns": [
    {"user_term": "account number", "table_name": "PACT_WORK",
     "column_name": "ACCOUNTNUMBER", "match_type": "normalized_name",
     "confidence": 0.99}
  ],
  "relationships": [],
  "unresolved_terms": [],
  "needs_grounding": false
}
```

4.3 `cc_schema_grounding_agent` - NEW agent replacing the `init` pair

What it does: resolve ambiguous business terms and literal values by inspecting a small, approved set of metadata/value profiles. It is not a second SQL generator. It exists because business questions often mention values such as “Tampa”, “open”, “high urgency”, or “complaint” without naming the physical filter column.

Why this replaces `cc_sql_init_generation_agent` + `cc_sql_init_execution_agent`

Today one agent generates exploratory SQL against `pact_metadata` and another executes it. Replace that pair with one grounding agent plus a restricted value-lookup tool. This removes LLM-generated exploratory SQL, reduces latency, and narrows the security surface.

- New tool: `command_center_value_lookup`.
- Input: `user_question`, `unresolved_terms`, and an allowlisted candidate list from `retrieved_metadata`.
- The tool must not accept arbitrary SQL. It accepts `table_name/column_name` candidates and a search value.
- The tool validates that each candidate is allowed for value lookup before querying.
- It returns exact/fuzzy matches, representative values, and confidence.
- If one candidate is clearly best, `grounding_output` contains the resolved table/column/value.

- If multiple candidates remain plausible, `grounding_output.needs_clarification=true` and the root asks the user.

instruction: |-

You are the Command Center Schema Grounding Agent.
Resolve only ambiguous business terms and filter values using `{retrieved_metadata}`.

Call `command_center_value_lookup` only when grounding is required.
Never generate final business SQL. Never join tables. Never invent mappings.
Inspect at most 10 approved candidate columns.

If one mapping is clearly supported, return it with confidence.
If ambiguity remains, set `needs_clarification=true`.

response_type: JSON

output_key: grounding_output

Example:

User: "show cases from Tampa"

Candidates: `PACT_WORK.OFFICE`, `PACT_WORK.LOCATION`, `PACT_WORK.REGION`

Value lookup result: `OFFICE` contains "Tampa"; other candidates do not.

Grounding output: `{"user_value": "Tampa", "column": "OFFICE", "table": "PACT_WORK", "resolved_value": "Tampa", "confidence": 0.98}`

4.4 cc_sql_generation_agent - generate only after schema grounding

What it should do: create one PostgreSQL read-only query from user intent plus approved grounded metadata. It should not perform schema discovery.

Exactly what to update

Remove rules that force the generator to guess when metadata is missing. The generator must return `NEEDS_CLARIFICATION` when the metadata layer explicitly reports ambiguity. Keep PostgreSQL syntax for the PoC; do not insert Oracle-only syntax yet.

- Use only tables/columns in `retrieved_metadata` and `grounding_output`.
- Never `SELECT *`.
- Only join when `AI_RELATIONSHIP_METADATA` marks the relationship verified.
- PostgreSQL PoC: `ILIKE` and `LIMIT` are valid. Use them where appropriate.
- Generate `SELECT` or `WITH...SELECT` only.
- Return SQL separately from parameter values.
- For non-aggregate detail queries, apply a configured row limit.
- Include `tables_used` and `columns_used` so the validator can compare intent to SQL.

instruction: |-

You are the Command Center PostgreSQL SQL Generation Agent.

USER QUESTION: `{user_question}`

APPROVED METADATA: `{retrieved_metadata}`

GROUNDED METADATA: `{grounding_output}`

Generate exactly one PostgreSQL `SELECT` or `WITH...SELECT` statement.

Use only approved tables/columns and only verified joins.

Never `SELECT *`. Never generate write operations.

Use `ILIKE` for case-insensitive text matching when appropriate.

Use `LIMIT` for detail-row limiting in the PostgreSQL PoC.

Return SQL and parameter values separately.

If metadata says the request is ambiguous, return `status=NEEDS_CLARIFICATION`.

response_type: JSON

output_key: sql_generation_output

4.5 cc_sql_validation_agent - strengthen into a real safety gate

What it should do: validate syntax, object allowlists, relationships, read-only safety and execution limits. LLM review alone is not enough; add a deterministic validation tool.

Exactly what to update

Register a new command_center_validate_sql tool. The agent should always call it. Validation must reject DML/DDL, unknown objects, unverified joins, multiple statements and restricted columns before execution.

- New tool: command_center_validate_sql.
- Parse PostgreSQL SQL using an approved SQL parser in the service layer.
- Allow only SELECT or WITH...SELECT.
- Reject INSERT, UPDATE, DELETE, MERGE, DROP, ALTER, TRUNCATE, CREATE, GRANT, EXECUTE and multiple statements.
- Extract referenced tables/columns and compare against metadata allowlists.
- Validate every join against AI_RELATIONSHIP_METADATA.
- Check restricted/PII flags, row limit, aggregate/group-by consistency, and query complexity thresholds.
- Return specific validation_errors to cc_root_agent for regeneration.

```
instruction: |-
  You are the Command Center SQL Validation Agent.
  Always call command_center_validate_sql with {sql_generation_output},
  {retrieved_metadata} and {grounding_output}.

  Do not execute SQL.
  If valid, return is_valid=true and validated_sql.
  If invalid, return is_valid=false plus precise validation_errors.

response_type: JSON
output_key: sql_validation_output
```

4.6 cc_sql_execution_agent - execution only, no generation

What it should do: execute exactly the validated SQL and return database output. The current prompt should be corrected because it calls this agent an expert SQL generation assistant.

Exactly what to update

Change the role text to “SQL Execution Agent”. It must refuse to run when is_valid is not true, must not modify SQL, and must pass bind/parameter values through the execution tool.

- Tool: command_center_run_sql (existing; harden it).
- Accept validated_sql plus parameter map/list.
- Use a read-only PostgreSQL service account / read-only transaction.
- Enforce statement timeout and server-side maximum rows.
- Return rows, column names, row_count, truncated flag, duration_ms, and error object.
- Do not summarize or repair SQL inside this agent.

```
instruction: |-
  You are the Command Center SQL Execution Agent.
  Execute only {sql_validation_output.validated_sql} when is_valid=true.
  Never modify or regenerate SQL.
  Always call command_center_run_sql with the validated SQL and parameters.
  Return the database result exactly in the execution output schema.
```

```
response_type: JSON
output_key: sql_query_result
```

4.7 cc_sql_summarization_agent - ground every answer in returned rows

What it should do: answer the original user question using only sql_query_result. It should never infer missing facts or state causes that were not measured.

Exactly what to update

Keep the current role, but pass the original user_question as well as sql_query_result. Add explicit handling for zero rows, truncated results, and database errors.

- Input: user_question + sql_query_result.
- No database tools.
- If zero rows, say that no matching records were found for the applied criteria.
- If result is truncated, say so.
- Preserve counts, percentages, units and date ranges.
- Do not expose internal agent prompts or hidden reasoning.
- Do not show SQL unless the user explicitly requests it.

```
instruction: |-
  You are the Command Center SQL Result Summarization Agent.
  Answer {user_question} using only {sql_query_result}.
  Do not invent missing facts or unsupported explanations.
  Handle zero rows, truncation and execution errors explicitly.
```

```
response_type: JSON
output_key: summary_response
```

5. Tool changes - exact contracts and implementation behavior

5.1 command_center_metadata_retrieval - EXISTING tool, must be upgraded

Today this tool should be treated as the most important component to improve. The SQL generator cannot compensate for weak schema linking. The upgraded tool must perform ranked metadata search and return structured candidates.

Recommended input contract:

```
{
  "user_question": "show account number for cases created yesterday",
  "max_tables": 5,
  "max_columns_per_table": 20,
  "include_relationships": true,
  "include_value_profiles": true
}
```

Recommended internal search order:

8. Normalize the user text and physical names by lowercasing and removing spaces, underscores, hyphens and punctuation.
9. Exact match physical name or normalized physical name.
10. Match display_name and synonyms from AI_COLUMN_METADATA.
11. Match AI_BUSINESS_GLOSSARY business_term and synonyms.
12. Search business_description and table descriptions using PostgreSQL full-text search; optionally add pg_trgm or embedding similarity later.
13. Score and rank candidate columns and tables.
14. Attach verified relationships and value profile summaries for top candidates.

15. Return unresolved terms and needs_grounding when confidence is insufficient.

Recommended output contract:

```
{
  "tables": [{"table_name": "PACT_WORK", "score": 94}],
  "columns": [{
    "user_term": "account number",
    "table_name": "PACT_WORK",
    "column_name": "ACCOUNTNUMBER",
    "display_name": "Account Number",
    "match_type": "normalized_name",
    "score": 99,
    "filterable": true,
    "selectable": true
  }],
  "relationships": [],
  "unresolved_terms": [],
  "needs_grounding": false
}
```

5.2 command_center_value_lookup - NEW restricted grounding tool

This is the replacement for LLM-generated exploratory SQL. It searches values only in allowlisted candidate columns and returns matches. It must never accept arbitrary SQL.

Input:

```
{
  "search_value": "Tampa",
  "candidates": [
    {"table_name": "PACT_WORK", "column_name": "OFFICE"},
    {"table_name": "PACT_WORK", "column_name": "LOCATION"},
    {"table_name": "PACT_WORK", "column_name": "REGION"}
  ],
  "max_values_per_column": 20
}
```

Output:

```
{
  "matches": [
    {
      "table_name": "PACT_WORK", "column_name": "OFFICE",
      "matched_value": "Tampa", "match_type": "case_insensitive_exact",
      "confidence": 0.98
    }
  ]
}
```

- Validate candidate table/column pairs against AI_COLUMN_METADATA where value_lookup_allowed=true.
- Use parameterized PostgreSQL queries; do not concatenate user text into SQL.
- Prefer exact case-insensitive match first, then prefix/contains match where safe.
- Return only small representative result sets.
- Never return sensitive values when PII classification disallows profiling.
- Log every value-lookup request for audit and tuning.

5.3 command_center_validate_sql - NEW deterministic validation tool

This tool is the execution safety gate. The validation agent should call it every time.

Input:

```
{
  "sql": "SELECT ...",
}
```



```

"parameters": {"p1": "Tampa"},
"allowed_tables": [...],
"allowed_columns": [...],
"verified_relationships": [...],
"database_dialect": "postgresql"
}

```

Output:

```

{
  "is_valid": true,
  "validated_sql": "SELECT ...",
  "errors": [],
  "warnings": [],
  "referenced_tables": ["PACT_WORK"],
  "referenced_columns": ["ACCOUNTNUMBER", "OFFICE"]
}

```

- Parse SQL instead of using regex alone.
- Reject multiple statements and write operations.
- Check table/column allowlists and join allowlists.
- Reject direct system/catalog access unless explicitly required.
- Enforce row-limit/query-cost policy.
- Optionally run PostgreSQL EXPLAIN (without ANALYZE) for complex queries after syntax/security checks.

5.4 command_center_run_sql - EXISTING tool, harden the contract

- Accept validated SQL plus parameter values separately.
- Execute only in a read-only transaction/service role.
- Apply statement timeout and max result rows at the tool layer, not just prompt level.
- Return structured DB errors (SQLSTATE, message) rather than free text.
- Return duration, row count and truncation metadata for observability.
- Keep dialect adapter boundary here so PostgreSQL execution can later be replaced with Oracle execution without changing upstream agent contracts.

6. Metadata enrichment - exact changes to implement

This is the main fix for natural-language matching. The physical column name should remain unchanged, but the metadata layer must add a semantic layer around it. The examples below use the command_center schema; change the schema prefix if your metadata tables live elsewhere.

Important compatibility note

Your current init agent queries a flattened pact_metadata object with sample_values and distinct_value_count. Do not break the PoC while refactoring: keep pact_metadata as a compatibility VIEW backed by the enriched metadata tables until the init agents are retired.

6.1 Enrich AI_TABLE_METADATA

Add table-level business context so retrieval can rank the right table before looking at individual columns.

```

ALTER TABLE command_center.ai_table_metadata
  ADD COLUMN IF NOT EXISTS business_domain text,
  ADD COLUMN IF NOT EXISTS business_description text,
  ADD COLUMN IF NOT EXISTS row_grain text,
  ADD COLUMN IF NOT EXISTS default_date_column text,
  ADD COLUMN IF NOT EXISTS synonyms text[] DEFAULT ARRAY[]::text[],

```

```
ADD COLUMN IF NOT EXISTS example_questions text[] DEFAULT ARRAY[]::text[],
ADD COLUMN IF NOT EXISTS agent_rules text,
ADD COLUMN IF NOT EXISTS enabled_flag boolean DEFAULT true;
```

Populate at minimum: what the table represents, one-row-per-what, primary business identifiers, default date field, and 3-10 business synonyms/example questions.

6.2 Enrich AI_COLUMN_METADATA

Add the semantic fields that let “account number” resolve to ACCOUNTNUMBER and “created date” resolve to PXCREATEDATETIME.

```
ALTER TABLE command_center.ai_column_metadata
ADD COLUMN IF NOT EXISTS display_name text,
ADD COLUMN IF NOT EXISTS normalized_name text,
ADD COLUMN IF NOT EXISTS business_description text,
ADD COLUMN IF NOT EXISTS synonyms text[] DEFAULT ARRAY[]::text[],
ADD COLUMN IF NOT EXISTS semantic_type text,
ADD COLUMN IF NOT EXISTS selectable_flag boolean DEFAULT true,
ADD COLUMN IF NOT EXISTS filterable_flag boolean DEFAULT true,
ADD COLUMN IF NOT EXISTS value_lookup_allowed boolean DEFAULT false,
ADD COLUMN IF NOT EXISTS pii_classification text DEFAULT 'UNCLASSIFIED',
ADD COLUMN IF NOT EXISTS agent_notes text;

UPDATE command_center.ai_column_metadata
SET normalized_name = regexp_replace(lower(column_name), '^[a-z0-9]+', '', 'g')
WHERE normalized_name IS NULL;
```

Why normalized_name matters:

```
normalize("ACCOUNTNUMBER") -> accountnumber
normalize("account number") -> accountnumber
normalize("account_number") -> accountnumber
normalize("account-number") -> accountnumber
```

This creates an immediate deterministic match without requiring embeddings.

6.3 Add AI_BUSINESS_GLOSSARY - NEW table

The glossary maps business terminology to technical objects. It should be curated with the Command Center/data-owner team and becomes the most reliable source for terms that are not obvious from column names.

```
CREATE TABLE IF NOT EXISTS command_center.ai_business_glossary (
  glossary_id bigserial PRIMARY KEY,
  business_term text NOT NULL,
  normalized_term text NOT NULL,
  definition text,
  synonyms text[] DEFAULT ARRAY[]::text[],
  target_table text,
  target_column text,
  semantic_type text,
  priority integer DEFAULT 100,
  enabled_flag boolean DEFAULT true,
  created_at timestamptz DEFAULT current_timestamp,
  updated_at timestamptz DEFAULT current_timestamp
);
```

Business term	Synonyms	Target
Account Number	account no, acct number, account #	PACT_WORK.ACCOUNTNUMBER
Created Date	creation date, created time, when created	PACT_WORK.PXCREATEDATETIME
Case Status	status, current status, case state	PACT_WORK.<validated status column>
Sentiment	customer sentiment, sentiment type	CITI_DATA_AI_INSIGHTS.<validated sentiment column>

6.4 Add AI_VALUE_PROFILE_METADATA - NEW table

Do not overload the column definition table with large samples. Store low-volume, approved profiling information separately. This powers grounding without querying the full business table for every question.

```
CREATE TABLE IF NOT EXISTS command_center.ai_value_profile_metadata (  
    table_name text NOT NULL,  
    column_name text NOT NULL,  
    sample_values jsonb,  
    distinct_value_count bigint,  
    null_count bigint,  
    min_value text,  
    max_value text,  
    profile_method text,  
    last_profiled_at timestamptz DEFAULT current_timestamp,  
    PRIMARY KEY (table_name, column_name)  
);
```

- Profile only columns marked value_lookup_allowed=true.
- Good candidates: status, country, office, region, category, channel, sentiment, urgency, application type.
- Do not profile free-text case descriptions, emails, names, account identifiers or other sensitive/high-cardinality fields.
- Refresh profiles on a schedule appropriate to the data; daily/weekly is usually enough for a PoC.

6.5 Strengthen AI_RELATIONSHIP_METADATA

Relationship metadata must be the only source of join authority. Same-name columns are not enough.

```
ALTER TABLE command_center.ai_relationship_metadata  
    ADD COLUMN IF NOT EXISTS join_type text DEFAULT 'INNER',  
    ADD COLUMN IF NOT EXISTS relationship_description text,  
    ADD COLUMN IF NOT EXISTS verified_flag boolean DEFAULT false,  
    ADD COLUMN IF NOT EXISTS verified_by text,  
    ADD COLUMN IF NOT EXISTS verification_date timestamptz,  
    ADD COLUMN IF NOT EXISTS confidence numeric(5,4),  
    ADD COLUMN IF NOT EXISTS enabled_flag boolean DEFAULT true;
```

Only relationships with verified_flag=true and enabled_flag=true should be exposed to SQL generation.

6.6 Rebuild AI_AGENT_METADATA_V / pact_metadata compatibility view

Expose a flattened search-friendly view containing table context, column semantics, glossary terms and value profile summaries. If the current init agents require pact_metadata, point pact_metadata at this view until they are removed.

```
CREATE OR REPLACE VIEW command_center.ai_agent_metadata_v AS  
SELECT  
    t.table_name,  
    t.business_domain,  
    t.business_description AS table_description,  
    t.row_grain,  
    t.default_date_column,  
    t.synonyms AS table_synonyms,  
    c.column_name,  
    c.display_name,  
    c.normalized_name,  
    c.business_description AS column_description,  
    c.synonyms AS column_synonyms,  
    c.semantic_type,  
    c.selectable_flag,
```

```

    c.filterable_flag,
    c.value_lookup_allowed,
    c.pii_classification,
    v.sample_values,
    v.distinct_value_count
FROM command_center.ai_table_metadata t
JOIN command_center.ai_column_metadata c
    ON c.table_name = t.table_name
LEFT JOIN command_center.ai_value_profile_metadata v
    ON v.table_name = c.table_name AND v.column_name = c.column_name
WHERE t.enabled_flag = true;

```

Optional temporary compatibility view:

```

CREATE OR REPLACE VIEW pact_metadata AS
SELECT table_name, column_name, sample_values, distinct_value_count,
       display_name, normalized_name, column_description, column_synonyms
FROM command_center.ai_agent_metadata_v;

```

7. How to enrich the metadata in practice

Do not try to manually document every column at once. Use a tiered enrichment process driven by real user questions.

16. Tier 1 - identify the 20-40 columns used by the first PoC questions. Populate display_name, description, synonyms, semantic_type and filter/select flags for these first.
17. Tier 2 - profile low-cardinality filter columns and load approved sample values/distinct counts into AI_VALUE_PROFILE_METADATA.
18. Tier 3 - create glossary entries for common business phrases from the Confluence KPI pages and user test questions.
19. Tier 4 - validate joins and populate AI_RELATIONSHIP_METADATA. Do not enable multi-table queries until this is complete.
20. Tier 5 - expand coverage based on failed queries and unresolved_terms captured in logs.

Recommended enrichment worksheet per column:

Field to fill	Example	How to decide
display_name	Account Number	Human-readable label
business_description	Account number associated with the case	Explain meaning, not datatype
synonyms	account no; acct number; account #	Terms users actually say
semantic_type	IDENTIFIER	IDENTIFIER / DATETIME / STATUS / CATEGORY / LOCATION / MEASURE / TEXT
selectable_flag	true	Can the user see it in results?
filterable_flag	true	Can the user filter on it?
value_lookup_allowed	false for account numbers	True only for safe categorical fields
pii_classification	CONFIDENTIAL or UNCLASSIFIED	Set with data governance
agent_notes	Use for case-level account lookup	Special generation guidance

Use your actual PoC questions as training data for metadata, not for the model

For every failed question, record the phrase the user used, the correct table/column mapping, and whether a glossary/synonym/value profile would have solved it. Update metadata, then rerun the same question. This creates a deterministic improvement loop without fine-tuning the model.

8. Natural-language schema linking algorithm

The metadata tool should score schema candidates before the SQL agent sees them. A simple deterministic scoring system is enough to materially improve the PoC.

Match type	Example	Suggested score
Exact physical name	ACCOUNTNUMBER	100
Normalized physical name	account number -> ACCOUNTNUMBER	98
Exact display name	Account Number	96
Glossary/synonym exact	acct number	94
Description/full-text strong match	case creation time	80-90
Trigram/semantic similarity	approximate business phrase	60-80

Confidence rule example: if the top candidate score is ≥ 90 and at least 10 points above the second candidate, accept it. Otherwise set `needs_grounding=true`. Tune these values with PoC tests; do not hard-code them forever.

9. Migration path from the current init agents

Do not remove `cc_sql_init_generation_agent` and `cc_sql_init_execution_agent` on day one. Replace them in two controlled steps.

21. Step A - enrich metadata tables and upgrade `command_center_metadata_retrieval` while keeping the existing init pair. Measure how often the init path is still required.
22. Step B - implement `command_center_value_lookup` and `cc_schema_grounding_agent`. Run both paths side-by-side on the same test questions and compare mappings.
23. Step C - change `cc_root_agent` to route to `cc_schema_grounding_agent` instead of the two init agents.
24. Step D - keep `pact_metadata` compatibility view temporarily so rollback is easy.
25. Step E - after grounding accuracy and latency meet the PoC target, disable `cc_sql_init_generation_agent` and `cc_sql_init_execution_agent` from the root flow. Do not delete configuration until the next stable release.

10. End-to-end examples after the changes

10.1 Example A - normal English column name

User: "show account number for cases created yesterday"

Metadata retrieval:

```
"account number" -> PACT_WORK.ACCOUNTNUMBER (normalized-name score 98)
"created" -> PACT_WORK.PXCREATEDATETIME (glossary/display score 96)
needs_grounding=false
```

SQL generation (PostgreSQL):

```
SELECT accountnumber, pxcreatedatetime
FROM pact_work
WHERE pxcreatedatetime >= date_trunc('day', current_timestamp) - interval '1 day'
  AND pxcreatedatetime < date_trunc('day', current_timestamp)
LIMIT 200;
```

Validation -> execution -> summary.

10.2 Example B - user value needs grounding

User: "how many cases are from Tampa?"

Metadata retrieval identifies candidate location-like columns:

```
OFFICE, LOCATION, REGION
needs_grounding=true
```

```
Schema grounding calls command_center_value_lookup("Tampa", candidates).
OFFICE contains Tampa -> confidence 0.98
```

```
SQL generation:
SELECT count(*) AS case_count
FROM pact_work
WHERE office ILIKE 'Tampa';
```

Validation -> execution -> summary.

10.3 Example C - multi-table request before relationships are verified

User: "show current case status with email history"

Metadata retrieval identifies PACT_WORK and EMT_WORK_HISTORY.
AI_RELATIONSHIP_METADATA contains no verified relationship.

Result:
SQL generation must not invent a join.
Root asks a clarification / explains that this combined query is not yet enabled.

After a verified relationship is added, the same request can proceed.

11. Recommended implementation sequence

Implement in this order

Metadata first, then grounding, then validation, then root orchestration. Rewriting prompts before fixing metadata retrieval will produce only incremental improvement.

Sprint / Step	Concrete work	Exit condition
1. Metadata foundation	Add semantic columns, normalized_name, glossary, value profile; populate Tier-1 fields	Top PoC questions resolve correct table/columns without exact physical names
2. Retrieval upgrade	Enhance command_center_metadata_retrieval scoring/output	Tool returns ranked candidates, confidence, unresolved_terms, needs_grounding
3. Grounding	Build command_center_value_lookup + cc_schema_grounding_agent	"Tampa/open/high urgency" style values map safely to correct columns
4. Validation	Build command_center_validate_sql and update validator prompt	Unsafe/unapproved SQL cannot reach execution
5. Agent prompts	Apply revised prompts and structured output contracts	All agents consume/produce standard state keys
6. Root orchestration	Implement conditional grounding and retry rules	Root skips unnecessary steps and controls retries
7. Retire init pair	Remove init agents from root flow after parallel testing	Grounding path meets accuracy/latency target
8. Expand metadata	Use failure logs to add glossary/synonyms/relationships	Coverage grows without prompt bloat

12. Test pack and acceptance criteria

Create at least 50 representative questions. Include exact physical names, normal English, abbreviations, misspellings, dates, categorical values, aggregates, empty-result cases and unsupported joins.

Test category	Example	Expected behavior
Physical column	Show ACCOUNTNUMBER	Direct metadata match
Normal English	Show account number	Normalized/glossary match to ACCOUNTNUMBER

Synonym	Show acct no	Synonym match
Date phrase	Cases created yesterday	Map to correct datetime field and PostgreSQL date predicate
Value grounding	Cases from Tampa	Ground Tampa to correct location-like column
Ambiguous term	Show status when multiple status columns exist	Ground or ask clarification
Unverified join	Case + email history without relationship	Do not generate join
Unsafe request	Delete failed records	Blocked before execution
PII request	Return restricted personal field	Blocked/masked based on metadata
Empty result	No matching records	Clear zero-row answer

13. Add observability so the PoC improves from real failures

Log these fields for every request. This is more valuable than adding more prompt instructions without evidence.

```
request_id
user_question
metadata_candidates + scores
unresolved_terms
grounding_invoked
grounding_result
generated_sql
validation_result + validation_errors
execution_duration_ms + row_count
summary_status
retry_count
final_success_flag
```

Weekly review: sort failures by unresolved term and add the smallest metadata fix (synonym, glossary entry, display name, description, value profile or relationship).

14. PostgreSQL PoC now, Oracle later - keep the migration boundary clean

The current PoC should continue generating PostgreSQL SQL. Do not mix Oracle syntax into the PoC prompts. Instead keep database_dialect in workflow state and isolate DB-specific behavior in the SQL generator rules, validator adapter and execution tool.

Concern	PoC: PostgreSQL	Later: Oracle
Case-insensitive match	ILIKE	UPPER(col) LIKE UPPER(:p)
Row limit	LIMIT n	FETCH FIRST n ROWS ONLY
Date/time	date_trunc, interval	TRUNC, INTERVAL / Oracle date functions
Bind handling	PostgreSQL driver adapter	Oracle driver adapter
SQL parser/validator	PostgreSQL dialect	Oracle dialect
Execution tool	PostgreSQL connection	Oracle connection

Migration rule

Keep the agent names and JSON contracts unchanged when moving to Oracle. Swap the dialect-specific generation rules, validator adapter and execution connection. This is why the tools should accept structured SQL + parameters rather than embedding database-driver details in prompts.

15. Definition of Done for the redesigned PoC flow

- A user can say “account number” and the metadata layer resolves ACCOUNTNUMBER without requiring the exact physical name.

- Common business terms from the KPI/Confluence pages are represented in AI_BUSINESS_GLOSSARY or column synonyms.
- Low-cardinality safe filter columns have value profiles; sensitive/high-cardinality columns do not.
- cc_schema_grounding_agent resolves ambiguous values through command_center_value_lookup; the old init pair is no longer in the primary flow.
- cc_sql_generation_agent only generates from approved grounded metadata.
- cc_sql_validation_agent always calls command_center_validate_sql and prevents unsafe/unapproved SQL from reaching execution.
- cc_sql_execution_agent cannot generate or modify SQL and runs only validated queries.
- cc_root_agent conditionally routes the workflow and enforces retry limits.
- At least 50 representative questions have documented expected schema mappings and results.
- Failure telemetry produces actionable metadata improvements each week.
- The PoC remains PostgreSQL-specific until acceptance; Oracle migration can occur by replacing dialect adapters rather than redesigning agents.

16. The shortest implementation checklist

26. ALTER the existing table/column/relationship metadata with the semantic fields in Section 6.
27. Create AI_BUSINESS_GLOSSARY and AI_VALUE_PROFILE_METADATA.
28. Populate Tier-1 metadata for the first PoC questions; create normalized_name for every column.
29. Upgrade command_center_metadata_retrieval to rank normal-language matches and return confidence/unresolved terms.
30. Create command_center_value_lookup and cc_schema_grounding_agent.
31. Create command_center_validate_sql and make validation mandatory.
32. Correct SQL generation, execution and summarization prompts using the contracts in Section 4.
33. Update cc_root_agent from fixed sequence to conditional supervisor logic.
34. Parallel-test the new grounding path against the existing init pair; then remove the init pair from the root flow.
35. Use logged failures to enrich glossary, synonyms, descriptions, value profiles and verified relationships.